

Chapter 1: Foundations of LLM Applications

Revised chapter with original wording, cleaner formatting, and source references.

Revision note: This version fixes broken PDF characters, rewrites generic explanations, adds original project examples, and includes references for key LLM concepts.

1.1 Introduction to LLM Applications

Large Language Model (LLM) applications are software systems that connect a user interface, business logic, and a language model to support tasks involving human language. Instead of relying only on fixed rules, these applications can interpret prompts, use context, and generate useful text responses.

A chatbot, writing assistant, coding helper, or document-review tool can all be designed as LLM applications. In each case, the application is responsible for collecting user input, preparing the request, sending it to a model, and presenting the model output in a useful way.

Project connection: In a plagiarism source finder, an LLM could explain why two sentences appear similar, while the search component provides source links and evidence for review.

1.2 What Is a Large Language Model?

A Large Language Model is an AI system trained on large collections of text so it can recognize language patterns and generate human-like responses. Stanford HAI describes LLMs as systems trained on massive text data to understand and generate language, while IBM explains that LLMs process prompts by converting text into smaller units and generating output one token at a time.

The important idea is that an LLM does not understand text in the same way a person does. It processes numerical representations of tokens and predicts likely continuations based on patterns learned during training.

1.3 How LLM Applications Work

Although real systems can include many components, the basic response flow is easy to understand: a user provides input, the input is prepared as tokens, the model processes those tokens, and the application returns output.

1. User Input	2. Tokenization	3. Model Processing	4. Output
The application receives a question, instruction, or document.	Text is split into tokens that the model can process.	The model uses learned patterns to estimate useful next tokens.	Tokens are converted back into readable text for the user.

Figure 1.1: A simplified LLM response flow.

1.3.1 User Input

The process begins when a user asks a question, uploads a document, or gives an instruction. The application may also add context, such as retrieved source material, user settings, or formatting requirements.

1.3.2 Tokenization

Before text reaches the model, it is divided into tokens. A token can be a word, part of a word, punctuation mark, or other text unit. OpenAI and IBM both describe tokenization as the step that breaks text into units a model can process.

1.3.3 Model Processing

After tokenization, the model processes the tokens using patterns stored in its parameters. The model estimates which tokens are likely to come next, guided by the input prompt and any context supplied by the application.

1.3.4 Output Generation

The model generates output by producing tokens step by step. The application then converts those tokens back into readable text and may format, filter, or validate the response before showing it to the user.

1.4 LLMs vs. Traditional Software Systems

Traditional software is usually built around explicit rules. If the same input enters the same deterministic function, the same output is expected. LLM-based applications are different because they generate language from learned patterns and can handle more flexible input.

Feature	Traditional Software	LLM-Based Application
Behavior	Follows explicit rules written by developers.	Generates responses using patterns learned from data.
Input handling	Works best with predictable inputs.	Handles many natural-language variations.
Output	Usually deterministic for the same input.	Can vary because generation is probabilistic.
Best use	Structured workflows, calculations, and strict business logic.	Language-heavy tasks such as summarization, drafting, and question answering.
Risk	Limited flexibility.	May produce inaccurate or uncited content if not reviewed.

Table 1.1: Comparing rule-based systems and LLM-based applications.

This difference is powerful, but it also creates responsibility. Developers should design guardrails, review workflows, citations, and user-facing explanations so generated content can be trusted and evaluated.

1.5 Training vs. Inference

LLM systems are often discussed in two phases: training and inference. Training is the learning phase, where a model is built or improved using large datasets. Inference is the usage phase, where the trained model responds to user input.

Aspect	Training	Inference
Purpose	Learn language patterns from large datasets.	Use a trained model to respond to a user request.
When it happens	Before deployment or during model improvement.	During application use.
Compute needs	Very high.	Lower than training, but still can be significant.
Input	Training data.	User prompt or application-provided context.
Output	Updated model parameters.	Generated text, code, summary, or decision support.

Table 1.2: Training and inference in LLM systems.

A simple analogy is studying and taking an exam. Training is similar to studying from many examples. Inference is similar to using that learned knowledge to answer a new question. The analogy is not perfect, but it helps separate model development from model usage.

1.6 Chapter Summary

This chapter introduced the foundation of LLM applications. An LLM application receives user input, prepares it for a model, generates or processes language, and returns useful output. Unlike traditional software, LLM systems are flexible with language but require careful review, citations, and responsible design.

For portfolio projects, the strongest LLM applications do more than call a model. They show clear user value, explain outputs, handle errors, cite sources when needed, and make the review process transparent.

References

IBM. (2026). What are large language models (LLMs)? IBM Think. <https://www.ibm.com/think/topics/large-language-models>

IBM. (2026). What is tokenization? IBM Think. <https://www.ibm.com/think/topics/tokenization>

OpenAI. (2026). What are tokens and how to count them? OpenAI Help Center.
<https://help.openai.com/en/articles/4936856-what-are-tokens-and-how-to-count-them>

Stanford Institute for Human-Centered Artificial Intelligence. (2026). What is a large language model (LLM)?
<https://hai.stanford.edu/ai-definitions/what-is-a-llm>

Google for Developers. (2026). Machine Learning Glossary. <https://developers.google.com/machine-learning/glossary>